内置组合数据类型

- **本章讨论的Python内置组合数据类型，主要包括以下五类：字符串、元组、列表、字典、集合。**
- 以下数据类型属于**不可变数据类型：字符串、元组。**
- 以下数据类型属于**可变数据类型：列表、字典、集合。**
- 以下三类数据类型属于**序列：字符串、元组、列表。**

序列

- 字符串是以字符组成的序列，元组是以任意数据类型组成的序列，它们都是不可变数据类型。
- 列表也是以任意数据类型组成的序列，但是它是可变数据类型。
- 序列有很多共性的属性和方法（下文详述），如：
    - 加法运算符、乘法运算符、成员运算符（in, not in）
    - 内置函数：len()、max()、min()
    - 索引（index）与切片（slice）

字符串

字符与字符串

- Python并不区分字符和字符串，即使只是一个字符，也被当做字符串处理。

单引号或双引号构造字符串

- 使用一对单引号或一对双引号，可以构造一个字符串，如 'helloworld'，"python"。
- 注意' '或" "本身只是一种表示方式，引号不是字符串的一部分。因此，字符串 'abc' 只有 a，b，c 这3个字符。
- 如果在字符串中需要出现双引号，则可以用单引号来构造字符串，反之亦然，如果'本身也是一个字符，那就可以用"....."括起来，比如"I'm OK"包含的字符是 I，'，m，空格，O，K 这6个字符。

转义字符

- 如果在字符串中相同时使用单引号和双引号，或者要表达一些特殊字符，则可以使用转义字符。

转义字符	说明
\n	换行符，将光标位置移到下一行开头。
\r	回车符，将光标位置移到本行开头。
\t	水平（水平）制表符，也即 Tab 键，一般相当于四个空格。
\v	纵向（垂直）制表符
\a	蜂鸣器响铃。注意不是喇叭发声，现在的计算机很多都不带蜂鸣器了，所以响铃不一定有效。
\b	退格（Backspace），将光标位置移到前一列。

转义字符	说明
\f	换页符
\'	单引号。如果在使用单引号作为限界的字符串中又要使用单引号，则可以在单引号前加 \ 。
\"	双引号。如果在使用双引号作为限界的字符串中又要使用双引号，则可以在双引号前加 \ 。
\	在字符串行尾的续行符，即一行未完，转到下一行继续写。
\ooo	3位八进制数 ooo 对应的字符，例如：'\141' 就是 'a'
\xhh	2位十六进制数 hh 对应的字符，例如：'\x61' 就是 'a'
\uhhhh	4位十六进制数 hhhh 表示的 Unicode 字符，例如：'\u4e2d' 就是 '中'
\\	反斜线。如果在字符串中要使用反斜线本身，则可以在反斜线前加 \ ，即连续使用二个反斜线。

换行符

- 相关的二个转义字符：回车符、换行符。
- 回车符：CR（Carriage Return），作用：使光标到行首。用符号'\r'表示，十进制ASCII代码是13，十六进制代码为0x0D。
- 换行符：LF（Line Feed），作用：光标下移一行。使用'\n'符号表示，ASCII代码是10，十六制为0x0A。
- 但是，实际如何表示换行，不同的操作系统有不同的方案。Unix下一般只有一个换行符LF(0x0A)表示换行("\n")，苹果 MAC OS系统则采用回车符CR(0x0D)表示下一行("\r")，windows则都是用0x0D和0x0A两个字符("\r\n")。

操作系统	换行符	换行符的转义字符	换行符的数值	说明
Unix/Linux	LF	\n	0x0A	换行
Mac	CR	\r	0x0D	回车
Windows/DOS	CR/LF	\r\n	0x0D0A	回车/换行

- 可以使用`os.linesep`查询当前操作系统使用的换行符。

```
import os
print(os.linesep)    # '\n'
```

原始字符串

- 如果不想使用转义字符，让Python在遇到转义字符时当普通字符处理，可以在字符串前加字母r。例如：

```
>>> print("hello\nworld")
hello
world
>>> print(r"hello\nworld")
hello\nworld
```

多行字符串

- Python中使用三引号表示多行字符串，即字符串中可以带换行。
- 所谓的三引号，可以是三对单引号，也可以是三对双引号。
- 多行字符串也常常使用在函数或类的开头，表示注释。
- 例如：

```
print('''
Python's triple quotes comes to the rescue by allowing strings to span
multiple lines, including verbatim NEWLINEs, TABs, and any other
special characters. The syntax for triple quotes consists of three
consecutive single or double quotes.
''')
```

字符串的比较运算

- 字符串的相等比较。使用==判断是否相等，使用!=判断是否不相等。相等的含义即两个字符串一模一样。
- 字符串不相等时比大小，规则是两个字符串的字符逐一对齐比较。字符比较大小的依据是它的Unicode编码，编码越大，字符在比较时就越大。如果两个字符串的前面部分一样，但是一个字符串较长，则较长者较大。例如：

```
>>> 'apple' < 'banana'  # True，因为双方的第一个字符 'a' < 'b'
True
>>> 'apple' < 'application'  # True，因为双方的第五个字符 'e' < 'i'
True
>>> 'apple' < 'applepie' # True，因为双方前五个字符相同，但是后者更长
True
```

可用于序列的函数和运算符（以字符串为例）

字符串与加法运算符

- 加法，形式为 `a+b` ，其中 a 和 b 都必须为字符串，结果为两个字符串相连。例如："hello"+"world" 将得到 "helloworld" ，注意两个字符串之间不会自动加空格。

字符串与乘法运算符

- 乘法，形为`s*n 或 n*s`，其中s为字符串，n为整数，结果为字符串s重复n遍，例如："hello"*3 将得到 "hellohellohello"，注意多个字符串之间不会自动加空格。

成员运算符 in, not in

- in 操作符可以判断一个字符或字符串是否包含在另一个字符串当中。

```
s = 'world'
print('wo' in s) # True
print('wd' in s) # False
print('e' in s) # False
print('e' not in s) # True
```

求长度 len()

- len()可以返回字符串长度，即包含字符的个数，包括空格、换行等符号。注意一个汉字算一个字符。

```
message = "Hello world!\n你好，世界！"
print(len(message)) # 19
```

找最大最小值 max(), min()

- max(string) 返回字符串中编码最大的字母
- min(string) 返回字符串中编码最小的字母

```
s = 'world'
print(max(s),min(s)) # w d
```

索引与切片（以字符串为例）

字符串索引

- index，索引，又叫下标，指字符串中每个字符的位置编号。
- 通过位置编号，可以随机访问字符串中的任何一个字符。
- 字符串中的每个字符，都有二个位置编号（索引、下标、index）。
- 一种编号方式是使用非负整数，假设一个长度为n的字符串，从左向右分别编号为0，1，……，n-1。这种编号方式与大多数语言（如C语言）类似。特别要注意编号从0开始！
- 另一种编号方式是使用负整数，从右向左分别编号为-1，-2，……，-n。这种编号方式是大多数其它语言没有的。
- 使用一对方括号来访问某一个字符，格式：字符串[索引]。

```
+---+---+---+---+---+---+
| p | y | t | h | o | n |
+---+---+---+---+---+---+
  0   1   2   3   4   5
 -6  -5  -4  -3  -2  -1
```

```
s = 'python'
# 从左向右依次显示s中的每个字符
for c in s:
    print(c)
# 从左向右依次显示s中的每个字符
for i in range(len(s)):
    print(s[i])
# 从右向左（反向）依次显示s中的每个字符
for i in range(-1,-len(s)-1,-1):
    print(s[i])
```

字符串切片

- 指定编号的起止位置，可以从一个字符串中切出一段字符串，叫做切片。
- 格式：字符串[起点:终点:步长]，如`s[a:b:k]`。步长k和第2个冒号可缺省，默认k=1。起点a和终点b也可以省略（其含义在下面介绍），但是注意第1个冒号不可省略，即使在省略了a或b的情况下。
- 注意：a和b中如果有负值索引，Python会自动转换成非负值索引再进行处理。
- 如果a和b越界，并不会导致错误。
- 设字符串s的长度len(s)=n，则常见的几种切片写法的含义说明如下。
- `s[a:b]`：取下标大于等于a但小于b的字符组成切片，即从下标a到b-1的字符。如果b<=a，则切片为空字符串。
- `s[a:]`：取下标大于等于a的字符组成切片，即从下标为a的字符到字符串s的结尾。
- `s[:b]`：取下标小于b的字符组成切片，即从下标0到b-1的字符。
- `s[:]`：取所有字符，即切片为字符串s的复本。
- `s[a:b:k]`
    - 当k为正值时，以间隔k取下标大于等于a但小于b的字符，即下标为a,a+k,a+2k,……的字符，且最后一个字符小于b。
    - 当k为负值时，以间隔k取下标小于等于a但大于b的字符，即下标为a,a+k,a+2k,……的字符，且最后一个字符大于b。
- `s[::-1]`：取字符串s反转所有字符顺序的后组成的字符串，即s的倒序。

```
+---+---+---+---+---+---+
| p | y | t | h | o | n |
+---+---+---+---+---+---+
  0   1   2   3   4   5
 -6  -5  -4  -3  -2  -1
```

```
s = 'python'
print(s[1:4]) # yth
print(s[-4:4]) # th
print(s[-2:-4]) # 空字符串
print(s[3:]) # hon
print(s[:-3]) # pyt
print(s[5:10]) # n
print(s[1:5:2]) # yh
print(s[:]) # python
print(s[::-1]) # nohtyp
```

字符串的不可变性

- 字符串的不可变性，不是指字符串不可以赋新值。
- 字符串可以整体赋新值，但是不可以通过索引和切片修改原字符串中的部分字符，这叫做字符串的不可变性。

```
s = 'python'
r = ''
for c in s:
    r = c + r   # 正确
print(r) # nohtyp
for i in range(len(s)//2):
    s[i] = s[-i-1] # 错误
```

元组（tuple）

- 与字符串类似，元组也是一种序列。元组以小括号为限界标志。
- 区别于字符串的是，元组可以含有任意类型的元素。例如： (1, "hello", True)

创建元组

- 创建元组，可以直接用带小括号的序列形式赋值，也可以使用tuple()函数。
- 创建空元组：t = ( ) 。空元组会被认为是 False，而非空元组为 True。
- 创建一个元素的元组：t = (1,) 。此时其中的逗号不可省略，否则会小括号的语义与改变运算优先级相混淆，系统会认为 t=(1) 是 t=1 之意。

```
t = (1, "hello", True)  # (1, "hello", True)
t = tuple('hello') # ('h', 'e', 'l', 'l', 'o')
t = tuple(range(1,10)) # (1, 2, 3, 4, 5, 6, 7, 8, 9)
t = () # () 空元组
t = tuple() # () 空元组
t = tuple(1,2,3) # 错误!
t = tuple(1) # 错误!
t = (1) # 语法上没有错误，但是t此时的值为一个整数，而不是元组
t = (1,) # (1,) 此时t为仅有一个元素的元组
```

使用元组

- 使用 len() 函数可以求出元组长度
- 使用 max(), min() 函数可以求出元组中的最大元素和最小元素，前提是元素的类型是可比较的。
- 使用 sum() 函数可以求元组所有元素之和，前提是元素的类型是可相加的。

```
t = (1, -2, 0, 3, -4)
print(len(t)) # 5
print(max(t)) # 3
print(min(t)) # -4
print(sum(t)) # -2
```

- in 和 not in 运算测试元组的元素成员性，通常用于创建条件。

```
if 6 in t:
    print("t has a six.")
```

- 元组的索引方式和字符串一样。正向索引从 0 到 len()-1，反向索引从 -1 到 -len() 。

```
+----+----+----+----+----+
|  1 | -2 |  0 |  3 | -4 |
+----+----+----+----+----+
   0    1    2    3    4
  -5   -4   -3   -2   -1
```

- 元组的切片和字符串也是一样的。[a:b] 取从索引为 a 到 b 之间（含a不含b）的元素。

```
# display one item through an index
print(t[1]) # -2
# display a slice
print(t[1:4]) # (-2, 0, 3)
```

- 使用连接运算符“+”可以将两个元组连接起来。

```
# concatenate two tuples
a = (1, 3, 5)
b = (2, 4, 6)
print(a+b) # (1, 3, 5, 2, 4, 6)
```

- 使用乘号，同样也可以得到元素重复多次的一个新元组。

```
chest = ("gold", "gems")
print(t*3) # ('gold', 'gems', 'gold', 'gems', 'gold', 'gems')
```

- 可以使用比较运算符 ==, !=, >, >=, <, <= 比较二个元组，比较方法是元素之间逐一比较。

```
t1 = ('ant','apple','banana')
t2 = ('ant','applepie','avatar')
print(t1 < t2) # True
```

元组的内置函数

查找 index()

tuple.**index**(*x*[, *start*[, *end*]])

返回元组中第一个值为 *x* 的元素的零基索引。未找到指定元素时，触发 `ValueError` 异常。可选参数 *start* 和 *end* 是切片符号，用于将搜索限制为元组的特定子序列。返回的索引是相对于整个元组的开始计算的，而不是 *start* 参数。

```
fruits = ('orange', 'apple', 'pear', 'banana', 'kiwi', 'apple', 'banana')
i = fruits.index('banana')
# 3
i = fruits.index('banana', i+1)  # Find next banana starting at position 4
# 6
```

计数 count()

tuple.**count**(*x*)

返回元组中元素 *x* 出现的次数。

```
fruits = ('orange', 'apple', 'pear', 'banana', 'kiwi', 'apple', 'banana')
a = fruits.count('apple')
# 2
b = fruits.count('tangerine')
# 0
```

列表（list）

- 列表和元组一样都是序列。上述元组的各种属性和操作，大部分也适用于列表。不同点在于，列表是可变数据类型。

创建列表

- 用方括号替代圆括号创建的就是列表

```
# 使用字面量直接为列表赋值
list1 = [] # 空列表
list2 = [1] # 与元组不同，一个元素也不必在最后加逗号，因为不会混淆
list3 = [2,3,4]
list4 = ['python', 3.14, 666, False, (3,6,9), [4,6,8]]

# 使用list()函数创建列表
list1 = list() # 空列表
list2 = list([2,4,6]) # [2,4,6]
list3 = list(('red','green','blue')) # ['red','green','blue']
list4 = list(range(3,6)) # [3,4,5]
list5 = list('python') # ['p', 'y', 't', 'h', 'o', 'n']

# 错误示例
list1 = list(1,2,3) # 错误，list()的函数必须是可迭代的，如字符串、元组、列表等
```

使用列表（与元组相同之处）

- 与元组一样，可以使用 len() 求列表的长度
- 与元组一样，可以使用 max() 、min()、sum() 求列表中元素的最大值、最小值和求和。
- 与元组一样，可以使用 in 和 not in 运算测试列表的元素成员性，
- 与元组一样，可以使用索引和切片，对列表中的部分元素进行截取。
- 与元组一样，可以使用加法运算符连接二个列表。**但是要注意只有相同的类型才能连接，要么全是列表要么全是元组。**
- 与元组一样，可以使用乘法运算符，将一个列表重复若干次。
- 与元组一样，可以使用比较运算符 ==, !=, >, >=, <, <= 比较二个列表。
- 与元组一样，可以使用内置函数 index() 和 count() 完成查找和计数。

```
>>> a = [1,2,3]
>>> b = (4,5,6)
>>> a+b
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can only concatenate list (not "tuple") to list
>>> a+list(b)
[1, 2, 3, 4, 5, 6]
```

得到列表拷贝

- list.**copy**() 返回列表的浅拷贝。相当于 `a[:]` 。
- `a[:]` ： 返回列表的拷贝
- list2 = list(list1) ： list2 是 list1 的拷贝
- 拷贝与原列表不再有关联，修改一方不会影响另一方。

```
>>> list1 = [1, 2, 3]
>>> list2 = list1.copy()
>>> list3 = list1[:]
>>> list4 = list(list1)
>>> list2
[1, 2, 3]
>>> list3
[1, 2, 3]
>>> list4
[1, 2, 3]
```

理解列表的可变性

- 列表和元组最大的区别就是：列表是可变的，可以修改。
- 这里的可变性，或者说是可修改，指的是可以通过索引、切片、列表方法等方式，向列表中增加、删除或替换部分元素。
- 注意，通过向元组变量或列表变量直接赋值的方法是整体替换，不是部分修改，不受可变性约束。

```
t = (1,3,5)
print(t[0]) # 1
t[0] = 7 # 错误！元组具有不可变性。
t = (7,8,9) # 正确，直接向元组变量赋值整体替换一个元组是可行的。

l = [2,4,6]
print(l[0]) # 2
l[0] = 7 # 正确，列表具有可变性
l = [7,8,9] # 正确，直接向列表变量赋值整体替换一个列表也是可行的。

t = ([1,3],2,4)
t[0][1] = 5 # 正确
print(t) # ([1, 5], 2, 4)
```

下面我们来看一下有哪些方法可以修改列表的内容。

通过索引对列表元素进行赋值

- 因为列表可变，可以直接给现有元素赋新值。
- 虽然索引方法可以修改列表中的元素，但是不能创建新元素，也不能修改不存在的元素。

```
a = [1, 3, 5, 7, 9]
a[1] = 6
print(a)
# [1, 6, 5, 7, 9]
```

通过切片对列表进行赋值

- 下面赋值语句会修改列表。因为将只有一个元素的列表赋值给了拥有两个元素的切片，所以列表的长度就会缩短1个单位

```
a = [1, 3, 5, 7, 9]
a[1:3] = [6]
print(a)
# [1, 6, 7, 9]
```

增加元素 append()、insert() 和 extend()

- 使用 list.append(x) 将元素 x 添加到列表末尾，列表长度加一。
- 使用 list.insert(i, x) 将元素 x 添加到列表中索引为 i 的位置，原来索引为 i的及其后面的所有元素向后移一位，列表长度加一。
- 使用 list.extend(*iterable*)，将可迭代对象的元素添加到 list 列表末尾。

```
list1 = [1,2,3]
list1.append(6)
print(list1)
# [1, 2, 3, 6]
list1.insert(1,8)
print(list1)
# [1, 8, 2, 3, 6]
list2 = [5,7,9]
list1.extend(list2)
print(list1)
# [1, 8, 2, 3, 6, 5, 7, 9]
```

删除元素 pop() 和 remove()

- 使用 list.remove(x) 删除列表中内容为x的元素。注意如果x不存在，则会出错，如果内容为x的元素不只一个，则只会删除第一个，想全部删除，应配合 while 循环语句。

```
list1 = [1,2,3,8,4,5,6,8,7]
if 9 in list1:
    list1.remove(9) # 如果没有判断是否存在就删除，此处就会报错。
while 8 in list1:
    list1.remove(8) # 配合循环才能把某个内容的元素删除干净
print(list1) # [1, 2, 3, 4, 5, 6, 7]
```

- 使用 x = list.pop([i])，可以删除列表中索引为 i 的元素，并且被删除的元素会放入变量 x 中。如果不指定 i，则删除列表中最后一个元素。此处的[]意为可有可无。

```
list1 = [1,2,3,8,4,5,6,8,7]
x = list1.pop() # x = 7
y = list1.pop(3) # y = 8
print(list1) # [1, 2, 3, 4, 5, 6, 8]
```

- 使用 del 语句，也可以达到删除列表中某个元素的目地。注意 del 是条独立的语句，不是列表的函数。del 甚至可以直接删除整个列表。

```
list1 = [1,2,3,8,4,5,6,8,7]
del list1[3]
print(list1)
# [1, 2, 3, 4, 5, 6, 8, 7]
del list1
print(list1)
# NameError: name 'list1' is not defined
```

清空列表 clear()

- 使用 list.clear() 函数，可以删除列表里的所有元素。

翻转列表 reverse() 和 reversed()

- 使用 list.reverse() 函数，可以将列表中的内容顺序翻转。这个函数将修改列表本身。
- 使用 reversed(list) 函数，可以将列表中的内容顺序翻转。这个函数不会修改列表本身。

```
list1 = [3,5,7]
list2 = list(reversed(list1)) # list1内容不变，翻转结果放list2中
print(list1)
# [3, 5, 7]
print(list2)
# [7, 5, 3]
list1.reverse() # list1的内容直接翻转
print(list1)
# [7, 5, 3]
```

列表排序 sort() 和 sorted()

- 使用 list.sort() 函数，可以将列表中的内容排序。这个函数将修改列表本身，函数无返回值。
- 使用 new_list = sorted(iterable) 函数，也可以将列表中的内容排序。内建函数 sorted 不是列表的内置函数，可以对所有可迭代的对象 iterable 进行排序操作。sorted 不会修改列表本身，而是返回一个新的排序后的列表。
- 可选的参数 reverse： 排序时默认为升序，加上 reverse=True 可改为降序。
- 可选的参数 key： 使用 key 参数接收一个函数来指定排序的逻辑。函数本身带有一个参数，用来指代可迭代对象中有待排序的对象，返回结果为一个具备自然顺序的值（可用来比较大小）。该返回值将被作为排序的依据。

```
list1 = [5,3,7]
list1.sort()
print(list1) # list1的内容直接排序
# [3, 5, 7]
list1.sort(reverse=True)
print(list1)
# [7, 5, 3]

list1 = [5,3,7]
list2 = sorted(list1) # list1内容不变，排序结果放list2中
print(list1)
# [5, 3, 7]
print(list2)
# [3, 5, 7]
list2 = sorted(list1,reverse=True)
print(list1)
# [5, 3, 7]
print(list2)
# [7, 5, 3]

a = [5, -3, -7]
b = sorted(a, key=abs)
print(b)
# [-3, 5, -7]

# lambda函数经常用于高阶函数中
a = [('Two',2),('One',1),('Three',3),('Four',4)]
b = sorted(a)
c = sorted(a, key=lambda x:x[1])
print(b)
# [('Four', 4), ('One', 1), ('Three', 3), ('Two', 2)]
print(c)
# [('One', 1), ('Two', 2), ('Three', 3), ('Four', 4)]

# 按照字符串长度而非字典顺序进行排序
d = sorted(['apple','ant','duck'],key=lambda s:len(s))
print(d)
# ['ant', 'duck', 'apple']
```

逆向循环序列

- 逆向循环序列时，先正向定位序列，然后调用 `reversed()` 函数：

```
for i in reversed(range(1, 10, 2)):
    print(i)
'''
9
7
5
3
1
'''
```

按指定顺序循环序列

- 按指定顺序循环序列，可以用 `sorted()` 函数，在不改动原序列的基础上，返回一个重新的序列：

```
basket = ['apple', 'orange', 'apple', 'pear', 'orange', 'banana']
for i in sorted(basket):
    print(i)
'''
apple
apple
banana
orange
orange
pear
'''
```

理解元组和列表的使用场景

- 列表包含元组的全部功能，而且更多，为什么还要用元组?
- 元组比列表快。
- 元组更适合常量。
- 有时必须用元组。

去除序列中的重复元素

- 使用 `set()` 去除序列中的重复元素。使用 `sorted()` 加 `set()` 则按排序后的顺序，循环遍历序列中的唯一元素：

```
basket = ['apple', 'orange', 'apple', 'pear', 'orange', 'banana']
for f in sorted(set(basket)):
    print(f)
'''
apple
banana
orange
pear
'''
```

例子：计算机游戏的最高得分表

```
# High Scores
# Demonstrates list methods
scores = []
choice = None
while choice != "0":
    print(
    """
    High Scores
    0 - Exit
    1 - Show Scores
    2 - Add a Score
    3 - Remove a Score
    4 - Sort Scores
    """
    )
    choice = input("Choice: ")
    print()

    # exit
    if choice == "0":
        print("Good-bye.")

    # list high-score table
    elif choice == "1":
        print("High Scores")
        for score in scores:
            print(score)

    # add a score
    elif choice == "2":
        score = int(input("What score did you get?: "))
        scores.append(score)

    # remove a score
    elif choice == "3":
        score = int(input("Remove which score?: "))
        if score in scores:
            scores.remove(score)
        else:
            print(score, "isn't in the high scores list.")
```

```
    # sort scores
    elif choice == "4":
        scores.sort(reverse=True)

    # some unknown choice
    else:
        print("Sorry, but", choice, "isn't a valid choice.")
```

例子：统计字母出现次数

- 题目描述
  - 输入：一段文字，其中没有换行，忽略其中非英文字母的其它字符，统计其中26个字母出现的次数，大小写字母认为是同一个字母。
  - 输出：出现次数最多的字母的出现次数，然后是这个字母（小写形式），一行输出一个。如果出现次数最多的字母不止一个，则按照字母表的顺序全部显示，也是一行一个。
- 输入输出样例

  输入

```
He says: "It was the best of times, it was the worst of times."
```

  输出

```
8
s
t
```

```
text = input().lower()
freq = [0] * 26

for c in text:
    if 'a' <= c <= 'z':
        freq[ord(c)-ord('a')] += 1

max = max(freq)
print(max)

max_index = -1
for i in range(freq.count(max)):
    max_index = freq.index(max,max_index+1)
    print(chr(max_index+ord('a')))
```

例子：打印杨辉三角形

- 输入整数n，打印前n行的杨辉三角形

```
             1
           1   1
         1   2   1
       1   3   3   1
     1   4   6   4   1
   1   5  10  10  5   1
```

```
n = int(input())

b = [1]
print(b)
for j in range(n-1):
    a = b
    a.insert(0,0)
    a.append(0)
    b = []
    for i in range(len(a)-1):
        b.append(a[i]+a[i+1])
    print(b)
```

列表相关的高级函数

随机乱序

- random.shuffle(list)，可以打乱list的顺序。

```
import random
list1 = list(range(10))
print(list1)
# [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
random.shuffle(list1)
print(list1)
# [3, 6, 4, 5, 9, 2, 0, 7, 8, 1]
```

同时取索引和值 enumerate()

- 在序列中循环时，用 `enumerate()` 函数可以同时取出位置索引和对应的值：

```
for i, v in enumerate(['tic', 'tac', 'toe']):
    print(i, v)
'''
0 tic
1 tac
2 toe
'''
```

列表与字符串的转换 split() 和 join()

- split() 功能：使用参数作为分割线将原始字符串拆分成若干个字符串并组织成列表返回。

```
s = "hello world and python"
a = s.split()
print(a)
# ['hello', 'world', 'and', 'python']
b = s.split('o')
print(b)
# ['hell', ' w', 'rld and pyth', 'n']
```

- join() 功能：将原始字符串填充到参数的每个字符或字符串之间组成新的字符串返回。

```
s = "python"
print("#".join(s))
# p#y#t#h#o#n
a = ['hello', 'world', 'and', 'python']
print("#".join(a))
# hello#world#and#python
```

过滤器filter()

filter() 函数用于过滤序列，过滤掉不符合条件的元素，返回值是迭代器对象，包含符合条件的元素。

- 该函数接收两个参数，第一个为函数，第二个为序列，序列的每个元素作为参数传递给函数进行判断，然后返回 True 或 False，最后将返回 True 的元素放到新列表中。 函数返回迭代器对象。
- filter() 方法的语法:

```
filter(function, iterable)
```

参数

- function -- 判断函数。
- iterable -- 可迭代对象。

以下展示了使用 filter 函数的实例：过滤出列表中的所有奇数

```
def is_odd(n):
    return n % 2 == 1

newlist = filter(is_odd, [1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
print(list(newlist))

# [1, 3, 5, 7, 9]
```

另一个例子：过滤仅包含小写字母的单词

```
names=["Alex","amanda","xiaowu"]

print(list(filter(lambda x:x.islower(),names)))

# ['amanda', 'xiaowu']
```

映射函数map()

使用 `map()` 会根据提供的函数对指定序列做映射。map() 函数语法：

`map(function, iterable, ...)`

- map() 函数接收的是两个参数，一个是函数名，另外一个是序列，其功能是将序列中的数值作为函数的参数依次传入到函数值中执行，然后再返回到列表中。返回值是一个迭代器对象。

```
a = list(map(lambda x: x ** 2, [1, 2, 3, 4, 5]))
print(a)
# [1, 4, 9, 16, 25]
```

- 可以看出，只要用map函数，就可以让列表中的每一个数都完成一次对函数参数的调用，并将结果返回到一个可迭代对象中。
- 可以通过 list(map()) 将map函数返回的迭代对象转化为列表

reduce()

在Python中，reduce 函数是一个用于对序列进行累积操作的函数。它通常用于将一个接受两个参数的函数应用于序列的所有元素，从而将它们缩减为单个值。例如，可以使用 reduce来累加或累乘一系列的数字。

- reduce函数第一个参数为函数，另一个参数为可迭代对象 Iterable Object（eg: list列表）。其返回值为一个值而不是迭代器对象，故其常用与叠加、叠乘等。
- reduce中的函数必须也要接收2个参数，执行时把前一个结果继续和序列的下一个元素做累积计算，其效果就是：
  `reduce(f, [x1, x2, x3, x4]) = f(f(f(x1, x2), x3), x4)`
- 使用 reduce() 函数时需要将其从 functools 中导入。

```
from functools import reduce
def func(x, y):
    return x*10 + y

reduce(func, [3,4,5,6])
# func(3,4) = 3*10+4 = 34
# func( func(3,4),5) = func(34, 5) = 34*10+5 = 345
# func( func( func(3,4), 5), 6) = 345*10 +6 = 3456

# 结果是：3456
```

- Map/Reduce常常一起配合使用，下面的例子是一个用Map/Reduce把str转换为int的函数：

```
from functools import reduce
```

```
DIGITS = {'0': 0, '1': 1, '2': 2, '3': 3, '4': 4, '5': 5, '6': 6, '7': 7, '8': 8, '9': 9}

def str2int(s):
    def fn(x, y):
        return x * 10 + y
    def char2num(s):
        return DIGITS[s]
    return reduce(fn, map(char2num, s))

print(str2int('123456789'))

# 结果是：123456789
```

匹配两个列表中的元素 zip()

- 同时循环两个或多个序列时，用 `zip()` 函数可以将其内的元素一一匹配：

```
>>> a = [1,2,3]
>>> b = [4,5,6]
>>> c = [4,5,6,7,8]
>>> zipped = zip(a,b)      # 返回一个对象
>>> zipped
<zip object at 0x103abc288>
>>> list(zipped)  # list() 转换为列表
[(1, 4), (2, 5), (3, 6)]
>>> list(zip(a,c))              # 元素个数与最短的列表一致
[(1, 4), (2, 5), (3, 6)]
```

序列常用方法总结

操作符或语句

方法	作用	是否适用于字符串	是否适用于元组	是否适用于列表
+	连接	是	是	是
*	重复	是	是	是
in	成员判定	是	是	是
not in	成员判定	是	是	是
[i]	索引	是（限读取，不可用于修改）	是（限读取，不可用于修改）	是
[a:b]	切片	是（限读取，不可用于修改）	是（限读取，不可用于修改）	是
del	删除	可删除整个字符串，但不可删除部分字符	可删除元组，但不可删除部分元素	是

通用函数（非内置）

方法	作用	是否适用于字符串	是否适用于元组	是否适用于列表
len(items)	求长度	是	是	是
max(items)	求最大值	是	是	是
min(items)	求最小值	是	是	是
sum(items)	求和	否	是	是
reversed(items)	翻转	是	是	是
sorted(items)	排序	是	是	是
map()	映射	是	是	是
zip()	匹配	是	是	是

元组或列表的内置函数

方法	作用	是否适用于列表	是否适用于元组
index(*x*[, *start*[, *end*]])	查找	是	是
count(x)	统计个数	是	是
copy()	得浅拷贝	是	否
append(x)	追加	是	否
insert(i, x)	插入	是	否
extend(*iterable*)	扩展	是	否
remove(x)	删除	是	否
pop([i])	弹出	是	否
clear()	清空	是	否
reverse()	翻转	是	否
sort()	排序	是	否

多维嵌套序列

- 利用嵌套序列（如：多维列表）组织和存储更为复杂的信息
- 列表和元组都可以存放任何类型数据，包含其他列表和元组
- 嵌套序列：序列中有序列
- 例如，全班期末成绩单：包含所有学生，每个学生包含所有成绩

创建嵌套序列

- 嵌套列表或元组的创建和普通列表和元组差不多
- 虽然可以用任意数量的列表和元组来创建嵌套序列，但最好还是使用一致的模式
- 嵌套超过两层并不好

```
nexted = ['first',('second','third'),['fourth','fifth','sixth']]
scores = [('Moe',1000),('Larry',1500),('Curly',3000)]
```

访问嵌套元素

- 与普通序列一样都是用索引
- 要访问下一层嵌套的一个元素，可以先把第一层元素赋值给变量
- 或者使用两重索引

```
scores = [('Moe',1000),('Larry',1500),('Curly',3000)]
print(scores[0]) # ('Moe', 1000)
a_score = scores[2]
print(a_score) # ('Curly', 3000)
print(a_score[0]) # Curly
```

例子：计算机游戏的最高得分表 2.0

```
# High Scores 2.
# Demonstrates nested sequences
scores = []
choice = None
while choice != "0":
    print(
    """
    High Scores 2.0
    0 - Quit
    1 - List Scores
    2 - Add a Score
    """
    )
    choice = input("Choice: ")
    print()
    # exit
    if choice == "0":
        print("Good-bye.")
    # display high-score table
    elif choice == "1":
        print("High Scores\n")
        print("NAME\tSCORE")
        for entry in scores:
            score, name = entry
            print(name, "\t", score)
    # add a score
```

```
    elif choice == "2":
        name = input("What is the player's name?: ")
        score = int(input("What score did the player get?: "))
        entry = (score, name)
        scores.append(entry)
        scores.sort(reverse=True)
        scores = scores[:5] # keep only top 5 scores
    # some unknown choice
    else:
        print("Sorry, but", choice, "isn't a valid choice.")
```

二维列表（矩阵）

```
matrix = [
    [1, 2, 3, 4, 5],
    [6, 7, 0, 0, 0],
    [0, 1, 0, 0, 0],
    [1, 0, 0, 0, 8],
    [0, 0, 9, 0, 3],
]
```

	[0]	[1]	[2]	[3]	[4]
[0]	1	2	3	4	5
[1]	6	7	0	0	0
[2]	0	1	0	0	0
[3]	1	0	0	0	8
[4]	0	0	9	0	3

```
matrix[0] is [1, 2, 3, 4, 5]
matrix[1] is [6, 7, 0, 0, 0]
matrix[2] is [0, 1, 0, 0, 0]
matrix[3] is [1, 0, 0, 0, 8]
matrix[4] is [0, 0, 9, 0, 3]

matrix[0][0] is 1
matrix[4][4] is 3
```

初始化矩阵

```
matrix = []
numberOfRows = eval(input("Enter the number of rows: "))
numberOfColumns = eval(input("Enter the number of columns: "))
for row in range(numberOfRows):
    matrix.append([])
    for column in range(numberOfColumns):
        value = eval(input("Enter an element: "))
        matrix[row].append(value)
```

或者

```
matrix = []
numberOfRows = eval(input("Enter the number of rows: "))
for row in range(numberOfRows):
    numbers = [ eval(x) for x in input().split() ]
    matrix.append(numbers)
```

打印矩阵

```
for row in range(len(matrix)):
    for column in range(len(matrix[row])):
        print(matrix[row][column],end=' ')
    print()
```

或者

```
for row in matrix:
    for value in row:
        print(value,end=' ')
    print()
```

例子：给选择题评分

```
# Students' answers to the questions
answers = [
  ['A', 'B', 'A', 'C', 'C', 'D', 'E', 'E', 'A', 'D'],
  ['D', 'B', 'A', 'B', 'C', 'A', 'E', 'E', 'A', 'D'],
  ['E', 'D', 'D', 'A', 'C', 'B', 'E', 'E', 'A', 'D'],
  ['C', 'B', 'A', 'E', 'D', 'C', 'E', 'E', 'A', 'D'],
  ['A', 'B', 'D', 'C', 'C', 'D', 'E', 'E', 'A', 'D'],
  ['B', 'B', 'E', 'C', 'C', 'D', 'E', 'E', 'A', 'D'],
  ['B', 'B', 'A', 'C', 'C', 'D', 'E', 'E', 'A', 'D'],
  ['E', 'B', 'E', 'C', 'C', 'D', 'E', 'E', 'A', 'D']]

# Key to the questions
keys = ['D', 'B', 'D', 'C', 'C', 'D', 'A', 'E', 'A', 'D']

# Grade all answers
for i in range(len(answers)):
    # Grade one student
    correctCount = 0
    for j in range(len(answers[i])):
        if answers[i][j] == keys[j]:
            correctCount += 1
    print("Student", i, "'s correct count is", correctCount)
```

字典（dict）

- dict，源自 dictionary。
- 列表和元组把信息组织成序列。而字典是将信息成对存储，就像字典中单词和解释成对使用。
- 字典中的每对数据由“键key”和“值value”组成，称为键值对。
- 一个字典中不能有相同的键，而且键必须是不可变类型的，即键可以是数字、字符串、元组等类型，但不能是列表、字典或集合等类型。
- 值是可变的不可变的都行，值也可以重复。
- 字典也是可变数据类型。

创建字典

- 字典项：一个键和一个值组成，中间有一个冒号。
- 字典：用逗号隔开的多个字典项（键值对），再用花括号包含它们。
- 可以直接用字面量定义一个字典，也可以使用 dict() 函数创建字典。

```
dict1 = {} # 空字典
dict2 = dict() # 空字典
tel1 = dict([('sape', 4139), ('guido', 4127), ('jack', 4098)])
# {'sape': 4139, 'guido': 4127, 'jack': 4098}
tel2 = dict(sape=4139, guido=4127, jack=4098)
# {'sape': 4139, 'guido': 4127, 'jack': 4098}
```

访问字典中的值

- 使用 dict[key] 可以取得字典中的值。
- 与索引的区别：索引是位置编号，字典的查找用的是键。
- 不能用值来查找键，就像真正的字典不能用解释来查找单词一样。

```
students = {'3021666666': 'Zhang San', '3022888888': 'Li Si'}
print(students['3021666666']) # Zhang San
print(students['3021000000'])
'''
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: '3021000000'
'''
```

- 在获取值之前先用in运算符对键进行测试。
- in运算符在字典上和在列表和元组上用法一样
- 注意in运算符测试是的键，不是值。

```
if '3021000000' in students:
    print(students['3021000000'])
else:
    print('I don\'t know.')
# I don't know.
```

- 通过 dict.get(key) 方法也可以获取值。
- 当键不存在又没有默认值时，get(key) 返回None，而不是抛出异常。
- get(key) 方法可以人为的增加一个当键不存在时的默认值。

```
students = {'3021666666': 'Zhang San', '3022888888': 'Li Si'}
print(students.get('3022888888')) # Li Si
print(students.get('3022000000')) # None
print(students.get('3022000000','I don\'t know.'))
# I don't know.
```

- 使用 dict.keys() 函数，得到一个由字典的键组成的序列。
- 使用 dict.values() 函数，得到一个由字典的值组成的序列。
- 使用 dict.items() 函数，得到一个由字典的键值对组成的序列，其中的每一个元素是一个由键和值组成的形如(键,值)的元组。

```
students = {'3021666666': 'Zhang San', '3022888888': 'Li Si'}

keys = tuple(students.keys())
# ('3021666666', '3022888888')
values = tuple(students.values())
# ('Zhang San', 'Li Si')
items = tuple(students.items())
# (('3021666666', 'Zhang San'), ('3022888888', 'Li Si'))

for k,v in students.items():
    print('The key',k,'has the value of',v)
'''
The key 3021666666 has the value of Zhang San
The key 3022888888 has the value of Li Si
'''
```

新增字典中的键值对

- 通过 dict[key] = value 创建新字典项，key是键，value是值。
- 不让用户添加重复项是一种保护措施，这是一个好的编程习惯。

```
student_no = input("Who do you want me to add (No.)? ")
if student_no not in students:
    name = input("\nWhat's the name? ")
    students[student_no] = name
    print("\n", student_no, "has been added.")
else:
    print("\nThat one already exists! Try modifying it.")
```

修改字典中的键值对

- 通过 dict[key] = value 修改字典项，key是键，value是值。
- 替换和添加是一样的代码，因此使用时要小心。

```
student_no = input("Who do you want me to modify (No.)? ")
if student_no in students:
    name = input("\nWhat's the new name? ")
    students[student_no] = name
    print("\n", student_no, "has been modified.")
else:
    print("\nThat one doesn't exist! Try adding it.")
```

删除字典中的键值对

- 使用 del dict[key] 可以删除指定key的键值对。
- 也可以使用 dict.pop(key) 删除指定key的键值对。
- 删除时需要注意键必须存在，否则会报错，所以要先检查一下。

```
student_no = input("Who do you want me to delete (No.)? ")
if student_no in students:
    del students[student_no]
    print("\nOkay, I deleted", student_no)
else:
    print("\nI can't do that!", student_no, "doesn't exist in the dictionary.")

student_no = input("Who do you want me to delete (No.)? ")
if student_no in students:
    name = students[student_no]
    print("\nOkay, I deleted", student_no, '. That is', name)
else:
    print("\nI can't do that!", student_no, "doesn't exist in the dictionary.")
```

- dict.popitem() 随机删除一个(key,value)对并以元组形式返回被删除的内容。

清空字典

- 使用 dict.clear() 可以清空一个字典。

例子：找到出现次数最多的单词

- 题目描述
  - 输入：一段文字，其中所有的非英文字母（如数字、标点符号、空格等）均视为单词之间的分隔符，大小写字母认为是同一个字母，统计其中所有单词出现的次数。这段文字中没有换行，不会输入空字符串。
  - 输出：出现次数最多的单词的出现次数，然后是这个单词（小写形式），一行输出一个。如果出现次数最多的单词不止一个，则对单词排序后按升序全部显示，也是一行一个，出现次数只在第一行输出一次即可，不要每个单词前都输出一遍。
- 输入输出样例
- 输入

```
It was the best of times, it was the worst of times.
```

- 输出

```
2
it
of
the
times
was
```

```
wordCounts = {} # Create an empty dictionary to count words

# Count each word in the line
line = input().lower()
```

```
# Replace punctuations with space
for ch in line:
    if not ch.isalpha():
        line = line.replace(ch, " ")

words = line.split() # Get words from each line
for word in words:
    if word in wordCounts:
        wordCounts[word] += 1
    else:
        wordCounts[word] = 1

pairs = list(wordCounts.items()) # Get pairs from the dictionary
items = [[x, y] for (y, x) in pairs] # Reverse pairs in the list
items.sort(reverse=True) # Sort pairs in items
values = list(wordCounts.values()) # Get values from the dictionary

m = max(values)
print(m)

max_words = []
for i in range(len(items)):
    if items[i][0] == m:
        max_words.append(items[i][1])
    else:
        break
max_words.sort()
for word in max_words:
    print(word)
```

例子：一个可编辑的词典

```
# Geek Translator
# Demonstrates using dictionaries
geek = {"404": "clueless. From the web error message 404, meaning page not found.",
"Googling": "searching the Internet for background information on a person.",
"Keyboard Plaque" : "the collection of debris found in computer keyboards.",
"Link Rot" : "the process by which web page links become obsolete.",
"Percussive Maintenance" : "the act of striking an electronic device to make it work.",
"Uninstalled" : "being fired. Especially popular during the dot-bomb era."}

choice = None

while choice != "0":

    print("""
Geek Translator
0 - Quit
1 - Look Up a Geek Term
2 - Add a Geek Term
3 - Redefine a Geek Term
4 - Delete a Geek Term
```

```
""")

    choice = input("Choice: ")
    print()

    # exit
    if choice == "0":
        print("Good-bye.")

    # get a definition
    elif choice == "1":
        term = input("What term do you want me to translate?: ")
        if term in geek:
            definition = geek[term]
            print("\n", term, "means", definition)
        else:
            print("\nSorry, I don't know", term)

    # add a term-definition pair
    elif choice == "2":
        term = input("What term do you want me to add?: ")
        if term not in geek:
            definition = input("\nWhat's the definition?: ")
            geek[term] = definition
            print("\n", term, "has been added.")
        else:
            print("\nThat term already exists! Try redefining it.")

    # redefine an existing term
    elif choice == "3":
        term = input("What term do you want me to redefine?: ")
        if term in geek:
            definition = input("What's the new definition?: ")
            geek[term] = definition
            print("\n", term, "has been redefined.")
        else:
            print("\nThat term doesn't exist! Try adding it.")

    # delete a term-definition pair
    elif choice == "4":
        term = input("What term do you want me to delete?: ")
        if term in geek:
            del geek[term]
            print("\nOkay, I deleted", term)
        else:
            print("\nI can't do that!", term, "doesn't exist in the dictionary.")

    # some unknown choice
    else:
        print("\nSorry, but", choice, "isn't a valid choice.")
```

字典常用方法总结

说明说明方法	作用
in	对键进行成员判定
not in	对键进行成员判定
[key]	由key取value
del dict[key]	删除
len(dict)	求字典长度
max(dict)	查找最大的键
min(dict)	查找最小的键
sum(dict)	键求和
dict.get()	由key取value
dict.keys()	得到keys序列
dict.values()	得到values序列
dict.items()	得到(key,value)序列
dict.pop(key)	删除key并返回value
dict.popitem()	随机删除一个(key,value)对并返回
dict.clear()	清空字典

集合（set）

- 集合与列表类似，可以使用它们存储一个元素集合。
- 但是，不同于列表，集合中的元素是不重复且不是按任何特定顺序放置的。
- 集合就像是一个只有key没有value的字典。

创建集合

- Python同时使用花括号创建字典和集合，而 {} 被用来表示一个空字典。为了创建一个空集合，只能使用 set() 。
- 注意集合中的元素没有顺序。

```
s1 = set() # 注意 {} 指的是空字典，不是空集合
s2 = {1, 3, 5} # 由字面量生成集合
s3 = set((2, 4, 6)) # 从元组生成集合
s4 = set([ x*2 for x in range(1,10)]) # 从列表生成集合
# s4 == {2, 4, 6, 8, 10, 12, 14, 16, 18}
s5 = set('python') # 从字符串生成集合
# {'t', 'h', 'o', 'y', 'n', 'p'}
```

集合常用操作

- add(e) 添加元素，因为集合中元素不重复，如果欲添加的元素已在集合中，并不会增加新元素。
- remove(e) 删除元素。如果删除的元素不存在，remove()会报错。
- len(), max(), min(), sum() 同样适用于集合。
- in, not in 判断元素是否在集合中。

```
>>> s1 = {1, 3, 5}
>>> s1.add(6)
>>> s1
{1, 3, 5, 6}
>>> len(s1)
4
>>> max(s1)
6
>>> min(s1)
1
>>> sum(s1)
15
>>> 3 in s1
True
>>> s1.remove(5)
>>> s1
{1, 3, 6}
```

集合运算

名称	操作符	内置函数	说明
相等	s1 == s2		判断两个集合是否包含相同的元素。
不等	s1 != s2		判断两个集合是否包含不同的元素。
真子集	s1 < s2		s1是s2的真子集，则s2包含s1中所有的元素，且包含s1中没有的元素。
子集	s1 <= s2	s1.issubset(s2)	s1是s2的子集，则s2包含s1中所有的元素。
真超集	s1 > s2		s1是s2的真超集，则s1包含s2中所有的元素，且包含s2中没有的元素。
超集	s1 >= s2	s1.issuperset(s2)	s1是s2的超集，则s1包含s2中所有的元素
并集	s1 | s2	s1.union(s2)	两个集合的并集，是一个包含这两个集合所有元素的集合。

名称	操作符	内置函数	说明
交集	s1& s2	s1.intersection(s2)	两个集合的交集，是一个包含了两个集合共同元素的集合。
差集	s1 - s2	s1.difference(s2)	两个集合set1和set2之间的差集，是一个包含了出现在set1但不出现在set2 的元素的集合。
异或（对称差）	s1 ^ s2	s1.symmetric_diffierence(s2)	两个集合之间的对称差（或称为异或）集合，是一个包含了除它们共同元素之外所有在这两个集合之中的元素。

```
set1 = {"green", "red", "blue", "red"} # Create a set
print(set1)
# {'red', 'blue', 'green'}

print("Is red in set1?", "red" in set1)
# Is red in set1? True

set2 = set([7, 1, 2, 23, 2, 4, 5]) # Create a set from a list
print(set2)
# {1, 2, 4, 5, 7, 23}
list2 = list(set2) # Obtain a list from a set
# [1, 2, 4, 5, 7, 23]

print("length is", len(set2)) # Use function len
# length is 6
print("max is", max(set2)) # Use max
# max is 23
print("min is", min(set2)) # Use min
# min is 1
print("sum is", sum(set2)) # Use sum
# sum is 42

# s1 == {'red', 'blue', 'green'}
set3 = set1 | {"green", "yellow"} # Set union
print(set3)
# {'yellow', 'green', 'red', 'blue'}

# s1 == {'red', 'blue', 'green'}
set3 = set1 - {"green", "yellow"} # Set difference
print(set3)
# {'red', 'blue'}

# s1 == {'red', 'blue', 'green'}
set3 = set1 & {"green", "yellow"} # Set intersection
print(set3)
# {'green'}

# s1 == {'red', 'blue', 'green'}
set3 = set1 ^ {"green", "yellow"} # Set exclusive or
```

```
print(set3)
# {'yellow', 'red', 'blue'}

print(set1 == {"green", "red", "blue"}) # Compare two sets
# True

set1.add("yellow")
print(set1)
# {'yellow', 'red', 'blue', 'green'}

set1.remove("yellow")
print(set1)
# {'red', 'blue', 'green'}
```

推导式（解析式）

- 推导式 comprehensions（又称解析式），是Python的一种独有特性，可以从一个数据序列构建另一个新的数据序列。
- 推导式（comprehensions）分为三种：
    - 列表推导式（list comprehension）
    - 字典推导式（dict comprehension）
    - 集合推导式（set comprehension）
- 没有元组推导式，类似的语法生成的是元组生成器表达式（generator expression）

列表推导式

- 列表推导式也称列表解析式，可以使用非常简洁的方式对列表或其他可迭代对象的元素进行遍历，再次计算，快速生成满足特定需求的新列表。
- 列表推导式的语法形式为：

```
[expression for expr1 in sequence1 if condition1
for expr2 in sequence2 if condition2
for expr3 in sequence3 if condition3
…
…
for exprN in sequenceN if conditionN]
```

- 列表推导式在逻辑上等价于一个循环语句，只是形式上更加简洁。
- 下面例子中的列表推导式，等价于下面的循环

```
# 列表推导式
alist = [ x*x for x in range(10) ]
print(alist)
# [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# 循环
alist = []
for x in range(10):
    alist.append(x*x)
print(alist)
# [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

- 下面这个列表推导式中有两个循环，其中第一个循环可以看作外循环，执行得慢；第二个循环可以看作内循环，执行得快。

```
# 列表推导式
vec = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
numbers = [ num for ele in vec for num in ele ]
print(numbers)
# [1, 2, 3, 4, 5, 6, 7, 8, 9]

# 循环
vec = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
numbers = []
for ele in vec:
    for num in ele:
        numbers.append(num)
print(numbers)
# [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

- 过滤不符合条件的元素。在列表推导式中可以使用if子句对列表中的元素进行筛选，只在结果列表中保留符合条件的元素。
- 示例：下面的代码用于从列表中，筛选出列表中大于0的数字，符合条件的元素组成新的列表。

```
alist = [-1, -2, 1, 2, 3, -4, -5, -6]
blist = [ i for i in alist if i>0 ]
print(blist)
# [1, 2, 3]
```

- 已知有一个包含一些同学成绩的字典，现在需要计算所有成绩的最高分、最低分、平均分，并查找所有最高分的同学，代码可以这样编写：

```
scores = {'Zhang': 45, 'Li': 78, 'Wang': 40, 'Zhou': 96, 'Zhao': 65, 'Sun': 90}
highest = max(scores.values())
lowest = min(scores.values())
average = sum(scores.values())/len(scores)
highest_person = [ name for name,score in scores.items() if score == highest ]
print (highest,lowest,average,highest_person)
# 96 40 69.0 ['Zhou']
```

- 查找列表中最大元素的所有位置。

```
from random import randint
x = [ randint(1,10) for i in range(10) ]
print(x) # [9, 1, 9, 1, 2, 8, 10, 10, 1, 6]
m=max(x)
print(m) # 10
max_index = [ index for index,value in enumerate(x) if value == m ]
print(max_index) # [6, 7]
```

- 同时遍历多个列表或可迭代对象。

```
# 推导式法
result = [ (x,y) for x in [1,2,3] for y in [3,1,4] if x!=y ]
print(result)
# [(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]

# 循环法
result = []
for x in [1,2,3]:
    for y in [3,1,4]:
        if x!=y:
            result.append((x,y))
print(result)
# [(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```

- 实现矩阵转置

```
matrix = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]]

# 方法一：嵌套了列表推导式的列表推导式
result = [[row[i] for row in matrix] for i in range(4)]

# 方法二：等价于上面的列表推导式的执行过程
result=[]
for i in range(len(matrix[0])):
    result.append( [row[i] for row in matrix])

# 方法三：把内层的列表推导式也展开
for i in range(len(matrix[0])):
    temp=[]
    for row in matrix:
        temp.append(row[i])
    result.append(temp)

# 方法四：使用内置函数zip()和list()来实现矩阵转置
result = list(map(list,zip(*matrix)))

# result == [[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
```

- 列表推导式中可以使用函数或复杂表达式

```
print([pow(v,2) for v in [2,3,4,-1] if v>0])
# [4, 9, 16]

print([v**2 if v%2==0 else v+1 for v in [2,3,4,-1] if v>0])
# [4, 4, 16]
```

- 列表推导式生成100以内的所有素数

```
prime = [p for p in range(2,100) if 0 not in [p%d for d in range(2,int(p**0.5) )]]
print(prime)
# [2, 3, 4, 5, 6, 7, 8, 9, 11, 13, 15, 17, 19, 23, 25, 29, 31, 35, 37, 41, 43, 47, 49, 53,
59, 61, 67, 71, 73, 79, 83, 89, 97]
```

字典推导式

- 跟列表推导式的用法是差不多，但将中括号改成大括号。

```
#快速转换字典键值位置
mca={'a':1, 'b':2, 'c':3, 'd':4}
dicts={v:k for k,v in mca.items()}
print(dicts)
# {1: ‘a’, 2: ‘b’, 3: ‘c’, 4: ‘d’}
```

集合推导式

- 功能：集合推导式跟列表推导式差不多，都是对一个列表的元素全部执行相同的操作，但集合是一种无重复无序的序列。
- 区别：跟列表推导式的区别在于：

  1.不使用中括号，使用大括号；2.结果中无重复；

```
squared={i*2 for i in [1,1,2]}
print(squared)
# {2, 4}
```

元组生成器表达式

```
a = [i*i for i in range(10)]
b = (i*i for i in range(10))
print(a)
# [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
print(b)
# <generator object <genexpr> at 0x7fb04a24edd0>
c = tuple(b)
print(c)
# (0, 1, 4, 9, 16, 25, 36, 49, 64, 81)
```

序列解包

- 序列解包是Python特有的语法，它将一个序列（或任何可迭代对象）解开，并将得到的值存储到一系列变量中。

```
>>> x, y, z = 1, 2, 3
>>> print(x, y, z)
1 2 3
```

- 一个有趣的例子是，利用序列解包实现交换两个或多个变量的值：

```
>>> x = 1
>>> y = 2
>>> x, y = y, x
>>> print(x, y)
2 1
>>>
```

- 一个常见的应用场合：一个函数或方法返回一个元组（或序列多其他可迭代对象），这时可以直接使用一组变量承接返回值。例如把字典的`popitem`方法的返回值解包到key、value两个便里中：

```
>>> scoundrel = {'name': 'Robin', 'girlfriend': 'Marion'}
>>> key, value = scoundrel.popitem()
>>> key
'girlfriend'
>>> value
'Marion'
```

- **需要格外注意的是：解包的序列包含的元素个数必须与你在等号左边列出的目标个数相同，否则Python将引发异常。**
- 然而一个方法或函数返回序列时，其长度通常是不可预测的，又或许我们只关系其中的某些值，这时**可以使用通配符（*）来接收多余的值。**

```
>>> a, b, *rest = (1, 2, 3, 4)  # 使用*rest接收多余的值
>>> rest
[3, 4]
>>>
>>> a, *rest, b = [1, 2, 3, 4] # *rest放在中间
>>> rest
[2, 3]
>>>
>>> a, *rest, b = [1, 4]  # 序列没有足够的值
>>> rest
[]
```

- 带星号的变量被赋予一个列表，包含了序列其他所有的值。
- 带星号的变量可以出现在任意位置，不一定在末尾。
- 当序列长度不足时，带星号变量被赋予一个空列表。

- 嵌套解包

```
>>> (a, b), (c, d) = (1, 2), (3, 4)
>>> a, b, c, d
(1, 2, 3, 4)
```

函数与可变数据类型

参数为可变数据类型

传递列表为函数参数

- 注意：实参以引用传递

```
def main():
    x = 1 # x represents an int value
    y = [1, 2, 3] # y represents a list

    m(x, y) # Invoke f with arguments x and y

    print("x is", x)       # x is 1
    print("y[0] is", y[0]) # y[0] is 5

def m(number, numbers):
    number = 6 # Assign a new value to number
    numbers[0] = 5 # Assign a new value to numbers[0]
    numbers = [7, 8, 9] # numbers does no longer refer to y

main() # Call the main function
```

将两维数组传给函数

```
def getMatrix():
    matrix = [] # Create an empty list

    numberOfRows = eval(input("Enter the number of rows: "))
    numberOfColumns = eval(input("Enter the number of columns: "))
    for row in range(numberOfRows):
        matrix.append([]) # Add an empty new row
        for column in range(numberOfColumns):
            value = eval(input("Enter a value and press Enter: "))
            matrix[row].append(value)

    return matrix

def accumulate(m):
    total = 0
    for row in m:
        total += sum(row)
```

```
    return total

def main():
    m = getMatrix() # Get an list
    print(m)

    # Display sum of elements
    print("\nSum of all elements is", accumulate(m))

main() # Invoke main function
```

默认参数为列表

```
def add(x, lst = []):
    if x not in lst:
        lst.append(x)

    return lst

def main():
    list1 = add(1)
    print(list1)    # [1]

    list2 = add(2)
    print(list2)    # [1, 2]

    list3 = add(3, [11, 12, 13, 14])
    print(list3)    # [11, 12, 13, 14, 3]

    list4 = add(4)
    print(list4)    # [1, 2, 4]

main()
```

怎么修改?

```
def add(x, lst = None):
    if lst == None:
        lst = []
    if not(x in lst):
        lst.append(x)

    return lst

def main():
    list1 = add(1)
    print(list1)    # [1]

    list2 = add(2)
```

```
    print(list2)    # [2]

    list3 = add(3, [11, 12, 13, 14])
    print(list3)    # [11, 12, 13, 14, 3]

    list4 = add(4)
    print(list4)    # [4]

main()
```

函数中返回列表与函数中修改列表

- 返回新列表，原列表不变
    - list2 = sorted(list1)
    - list2 = reversed(list1)
- 直接修改原列表
    - list1.sort()
    - list1.reverse()

从函数中返回列表

```
def reverse1(lst):
    result = []
    for element in lst:
        result.insert(0,element)
    return result

list1 = [1, 2, 3, 4, 5, 6]
list2 = reverse1(list1)
print(list1) # [1, 2, 3, 4, 5, 6]
print(list2) # [6, 5, 4, 3, 2, 1]
```

直接修改原列表

```
def reverse2(lst):
    for i in range(len(lst)//2):
        lst[i], lst[-i-1] = lst[-i-1], lst[i]

list1 = [1, 2, 3, 4, 5, 6]
list2 = reverse2(list1)
print(list1) # [6, 5, 4, 3, 2, 1]
print(list2) # None
```

有什么问题?

```
def reverse3(lst):
    result = []
    for element in lst:
        result.insert(0,element)
    lst = result

list1 = [1, 2, 3, 4, 5, 6]
list2 = reverse3(list1)
print(list1) # [1, 2, 3, 4, 5, 6] ???
print(list2) # None
```

怎么修改?

```
def reverse4(lst):
    result = []
    for element in lst:
        result.insert(0,element)
    lst.clear()
    lst.extend(result)

list1 = [1, 2, 3, 4, 5, 6]
list2 = reverse4(list1)
print(list1) # [6, 5, 4, 3, 2, 1]
print(list2) # None
```